KG COLLEGE OF ARTS AND SCIENCE

Autonomous Institution | Affiliated to Bharathiar University

Accredited with A++ Grade by NAAC

ISO 9001:2015 Certified Institution

KGiSL Campus, Saravanampatti, Coimbatore - 641 035

LAB MANUAL

BUSINESS INTELLIGENCE LAB

B.Sc. CS (AI & DS)

2024 BATCH

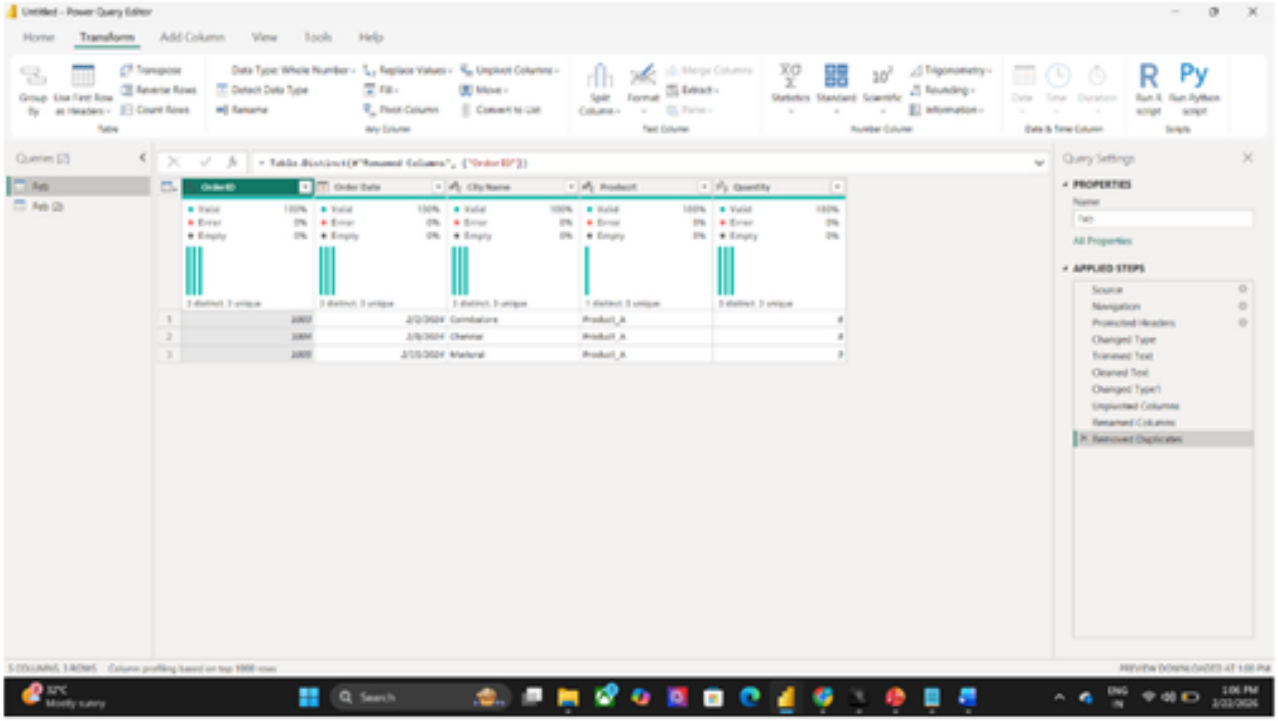
1.

Aim:To build a Power Query pipeline that combines multiple monthly files from a folder, standardizes the data, unpivots wide tables, removes duplicates, assigns correct data types, and prepares a data quality summary.

Algorithm (12 Points):

1. **Open Excel / Power BI and choose Get Data → From Folder.**
2. **Select the folder containing Jan_Sales and Feb_Sales files.**
3. **Click Combine & Transform to merge the monthly files.**
4. **Rename column headers to standard names (order_id, order_date, city).**
5. **Use Trim and Clean to standardize city text values.**
6. **Convert columns to proper data types (date, text, whole number).**
7. **Select Product_A, Product_B, Product_C columns.**
8. **Apply Unpivot Columns to convert wide table into long format.**
9. **Rename columns as product and quantity.**
10. **Remove duplicate records using order_id.**
11. **Duplicate the query and apply Group By → Count Rows for quality check.**
12. **Prepare a Data Quality Summary and load the final cleaned table.**

OUTPUT:



2.

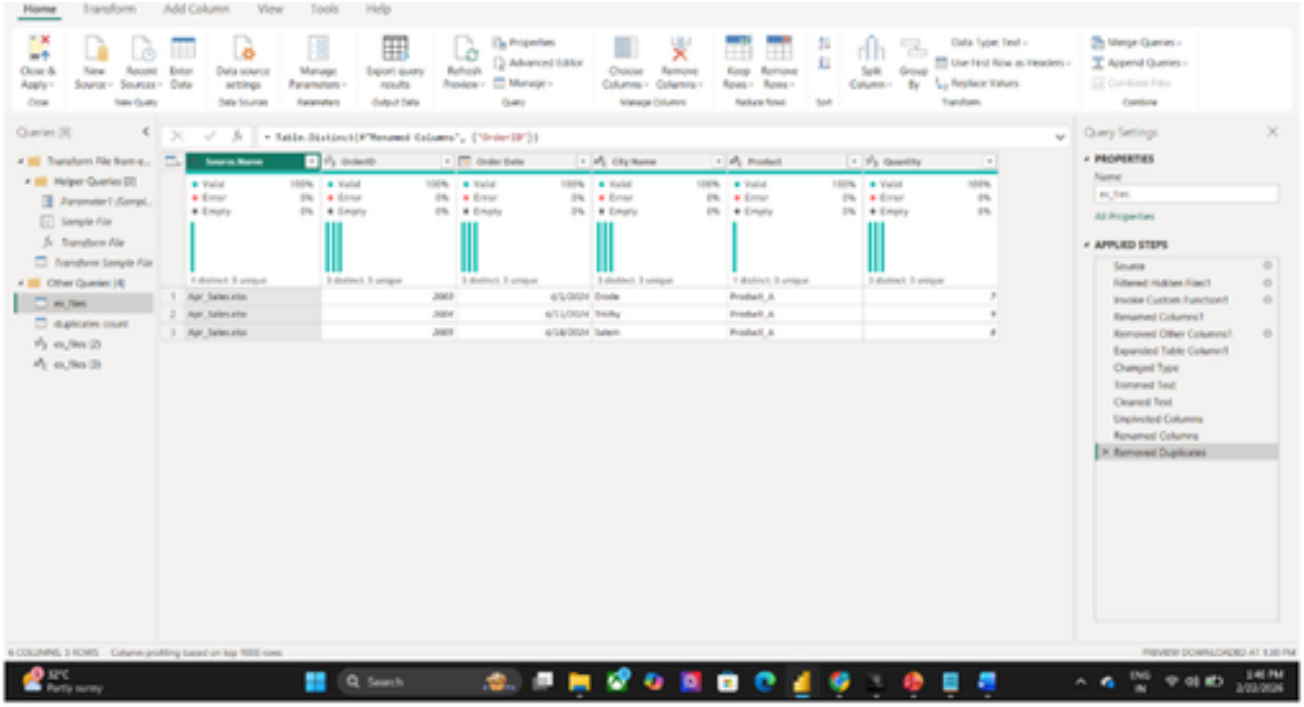
Aim:To perform an advanced Power Query transformation by combining quarterly files from a folder and applying data standardization, unpivoting, de-duplication, type correction, and data quality validation.

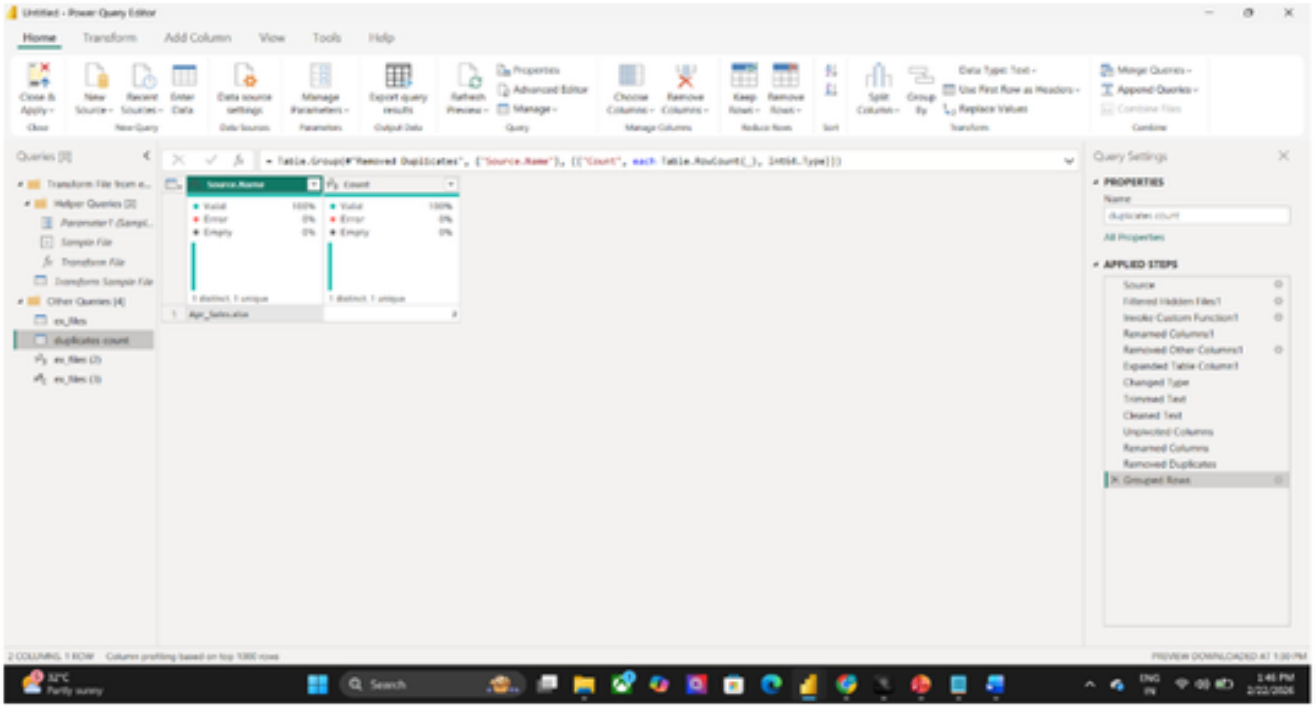
Algorithm (12 Points):

1. Open Excel / Power BI and select **Get Data → From Folder**.
2. Choose the folder containing **Mar_Sales** and **Apr_Sales** files.
3. Use **Combine & Transform** to append the files.
4. Standardize column names into lowercase meaningful names.
5. Apply **Trim** and **Clean** on city column to remove spaces.
6. Assign correct data types for all columns.
7. Select product columns and perform **Unpivot Columns**.
8. Rename Attribute and Value columns to product and quantity.
9. Remove duplicate records based on **order_id**.

10. Verify no missing or invalid quantity values.
11. Create a duplicate query and perform **Group By** for row count.
12. Create and load a **Data Quality Summary** table.

OUTPUT:





3.

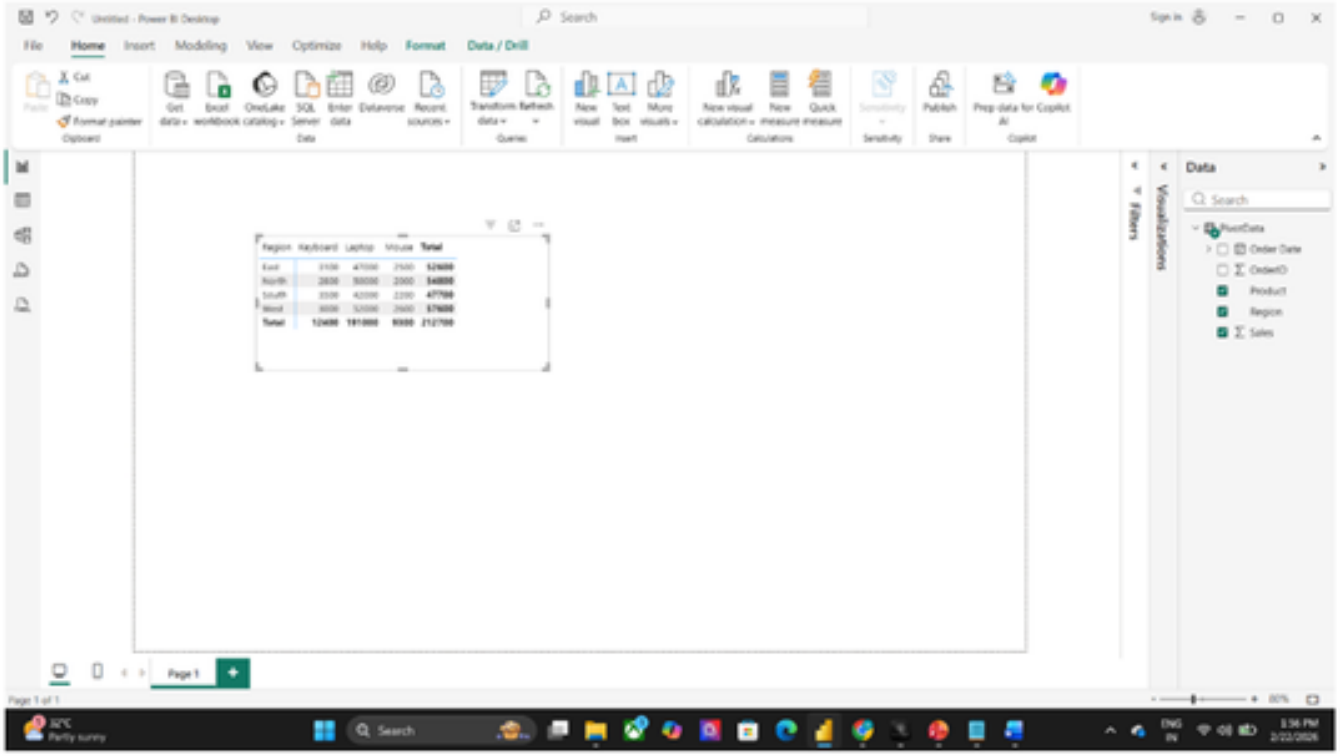
Aim:

To create a Pivot Table–style report in **Microsoft Power BI Desktop** using the Matrix visual to analyze sales by Region and Product.

Algorithm (12points):

1. Open **Microsoft Power BI Desktop**.
2. Click **Home → Get Data → Excel**.
3. Select the file **PowerBI_Pivot_Practice.xlsx** and load the data.
4. Go to **Report View**.
5. Click on the **Matrix** visual from the Visualizations pane.
6. Drag the **Region** field into the **Rows** section.
7. Drag the **Product** field into the **Columns** section.
8. Drag the **Sales** field into the **Values** section.
9. Ensure the aggregation is set to **Sum of Sales**.
10. Enable **Subtotals and Grand Totals** from the Format pane.
11. Sort the matrix in descending order based on total sales.
12. Observe and analyze the pivot-style summary of sales by region and product.

OUTPUT:



4.

Aim:

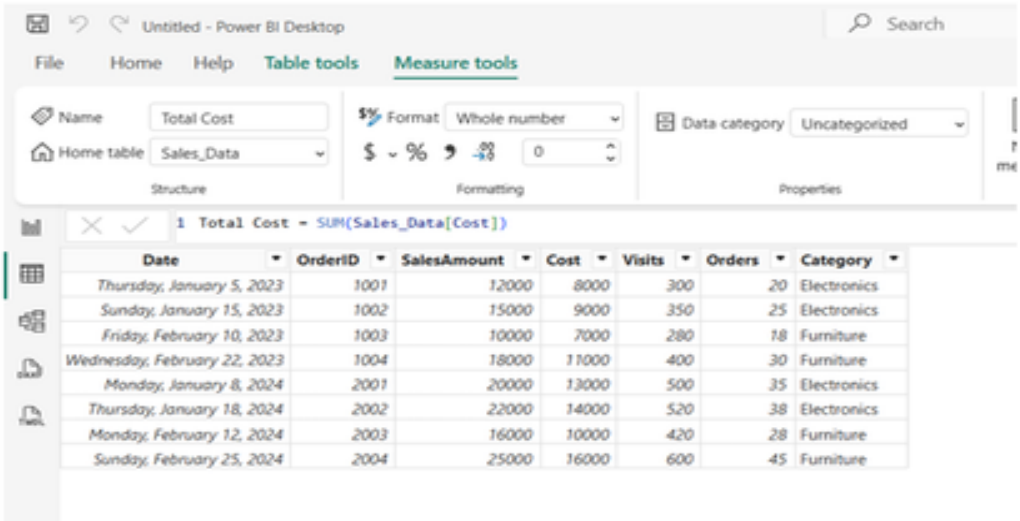
To create essential **DAX measures** in **Microsoft Power BI Desktop** for sales analysis, including revenue, cost, profit, conversion metrics, and time-intelligence calculations (YTD and YoY).

Algorithm (12 points):

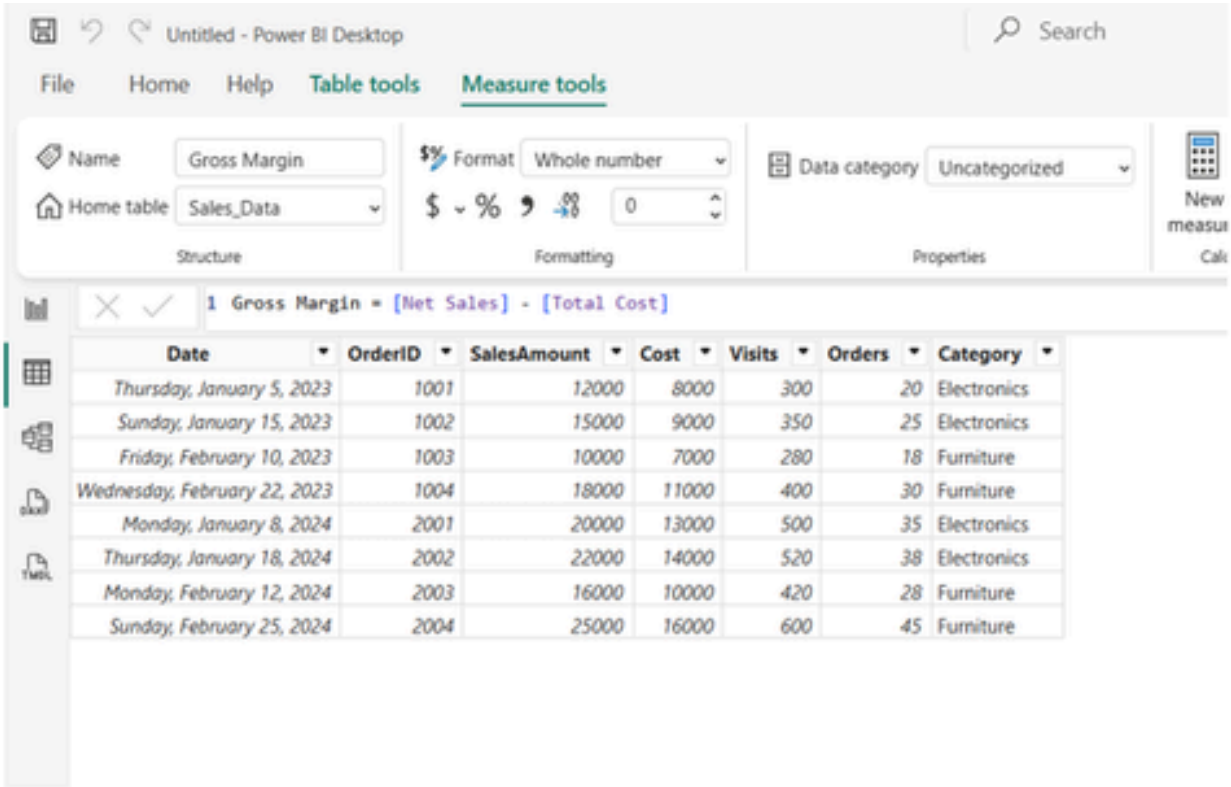
1. Open **Microsoft Power BI Desktop** and load the Sales dataset using **Home → Get Data**.
2. Go to **Report View** and open the **Fields** pane.
3. Right-click the **Sales** table and choose **New Measure**.
4. Create **Net Sales** measure using SUM(Sales[SalesAmount]).
5. Create **Total Cost** measure using SUM(Sales[Cost]).
6. Create **Gross Margin** using [Net Sales] - [Total Cost].
7. Create **GM %** using DIVIDE([Gross Margin], [Net Sales], 0) and format as Percentage.
8. Create **Total Orders** using DISTINCTCOUNT(Sales[OrderID]).
9. Create **Total Visits** using SUM(Sales[Visits]) and **Conversion Rate %** using Orders ÷ Visits.
10. Create a **Date Table** using CALENDAR and relate it to Sales[Date].

11. Create **Net Sales YTD** using TOTALYTD([Net Sales], DateTable[Date]).
12. Create **Net Sales YoY %** and **Contribution %** using time-intelligence and ALL function for total share calculation.

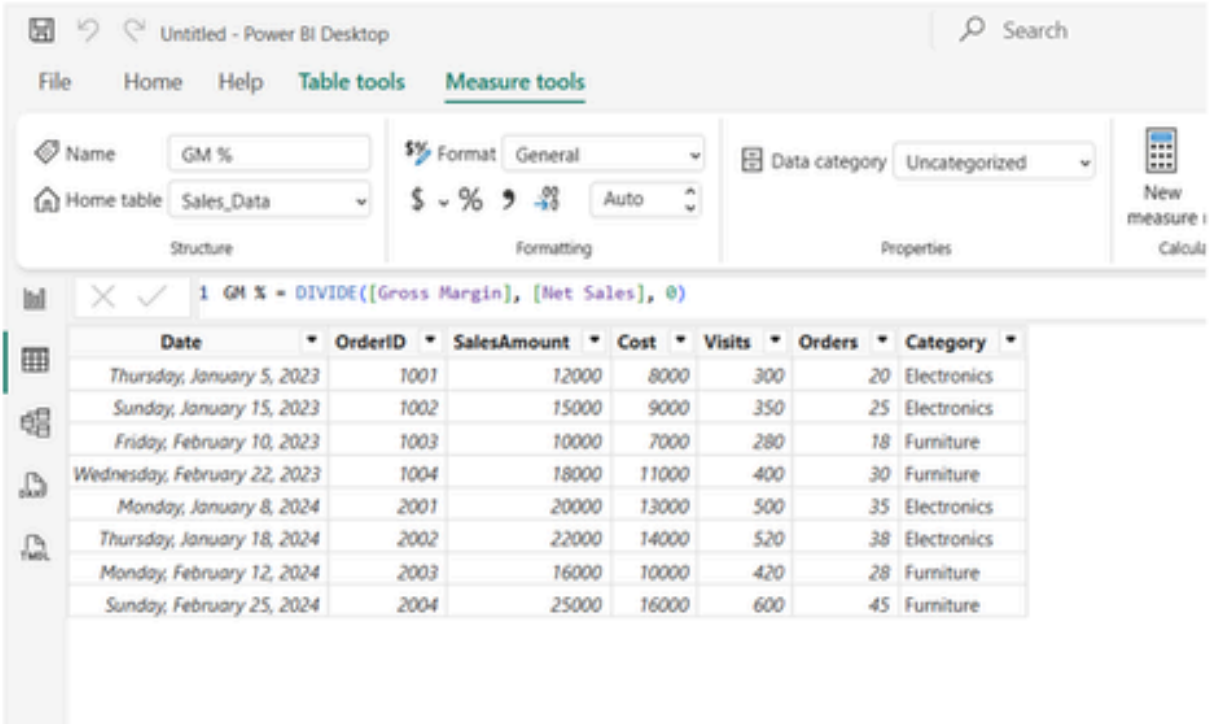
OUTPUT:



Date	OrderID	SalesAmount	Cost	Visits	Orders	Category
Thursday, January 5, 2023	1001	12000	8000	300	20	Electronics
Sunday, January 15, 2023	1002	15000	9000	350	25	Electronics
Friday, February 10, 2023	1003	10000	7000	280	18	Furniture
Wednesday, February 22, 2023	1004	18000	11000	400	30	Furniture
Monday, January 8, 2024	2001	20000	13000	500	35	Electronics
Thursday, January 18, 2024	2002	22000	14000	520	38	Electronics
Monday, February 12, 2024	2003	16000	10000	420	28	Furniture
Sunday, February 25, 2024	2004	25000	16000	600	45	Furniture



Date	OrderID	SalesAmount	Cost	Visits	Orders	Category
Thursday, January 5, 2023	1001	12000	8000	300	20	Electronics
Sunday, January 15, 2023	1002	15000	9000	350	25	Electronics
Friday, February 10, 2023	1003	10000	7000	280	18	Furniture
Wednesday, February 22, 2023	1004	18000	11000	400	30	Furniture
Monday, January 8, 2024	2001	20000	13000	500	35	Electronics
Thursday, January 18, 2024	2002	22000	14000	520	38	Electronics
Monday, February 12, 2024	2003	16000	10000	420	28	Furniture
Sunday, February 25, 2024	2004	25000	16000	600	45	Furniture



Date	OrderID	SalesAmount	Cost	Visits	Orders	Category
Thursday, January 5, 2023	1001	12000	8000	300	20	Electronics
Sunday, January 15, 2023	1002	15000	9000	350	25	Electronics
Friday, February 10, 2023	1003	10000	7000	280	18	Furniture
Wednesday, February 22, 2023	1004	18000	11000	400	30	Furniture
Monday, January 8, 2024	2001	20000	13000	500	35	Electronics
Thursday, January 18, 2024	2002	22000	14000	520	38	Electronics
Monday, February 12, 2024	2003	16000	10000	420	28	Furniture
Sunday, February 25, 2024	2004	25000	16000	600	45	Furniture

5.

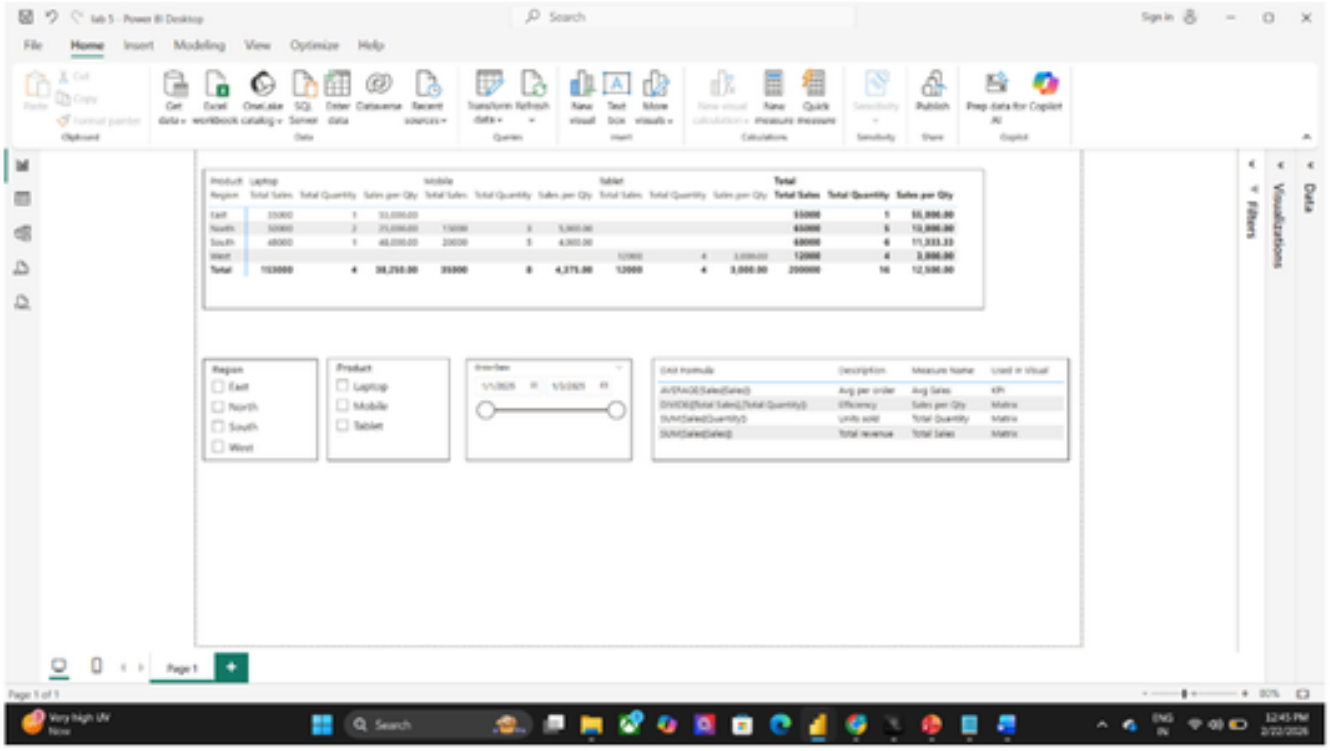
Aim:

To create and validate DAX measures in **Microsoft Power BI Desktop** using a Matrix (pivot-style) visual with slicers, and to document the measures using a Measure Catalogue table for reporting and export.

Algorithm (12 points):

1. Create an Excel file **SalesData.xlsx** with the given sales dataset.
2. Open **Microsoft Power BI Desktop** and load the Excel file using **Home → Get Data**.
3. Load the **Sales** sheet and move to **Report View**.
4. In the Fields pane, right-click the **Sales** table and select **New Measure**.
5. Create the measure **Total Sales** using SUM(Sales[Sales]).
6. Create the measure **Total Quantity** using SUM(Sales[Quantity]).
7. Create **Avg Sales** and **Sales per Qty** using AVERAGE and DIVIDE.
8. Insert a **Matrix** visual and place Region in Rows, Product in Columns, and all measures in Values.
9. Insert **Slicers** for Region, Product, and OrderDate to validate dynamic recalculation of measures.
10. Use **Home → Enter Data** to create a new table named **Measure Catalogue**.
11. Add columns for Measure Name, DAX Formula, Description, and Used In Visual, then load the table.
12. Display the Measure Catalogue using a Table visual and export it to Excel for documentation.

OUTPUT:



6.

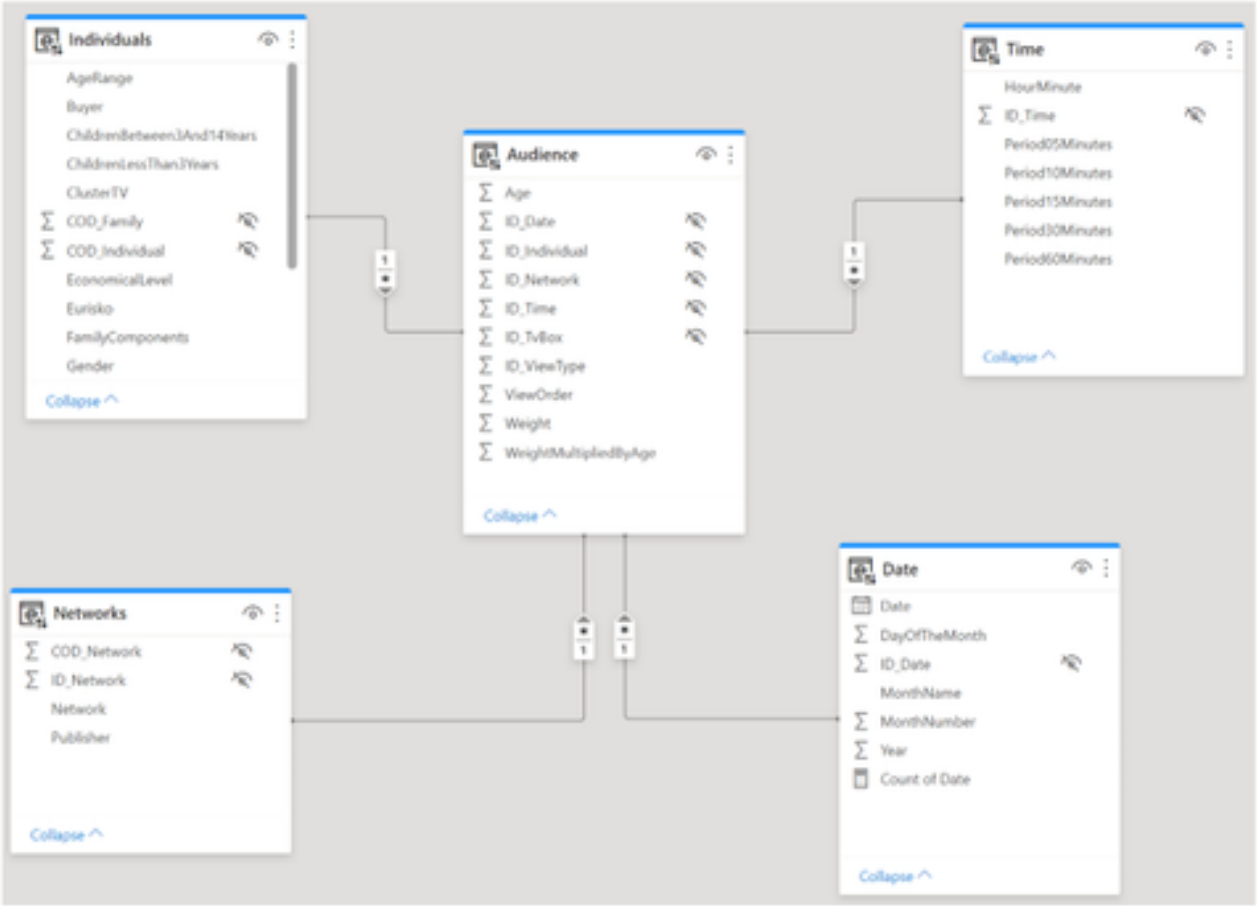
Aim:

To design a **Star Schema data model** in **Microsoft Power BI Desktop** by loading fact and dimension tables, and creating proper relationships in Model View for efficient reporting.

Algorithm (12 points):

1. Create Excel sheets for **Fact_Sales**, **Dim_Product**, **Dim_Customer**, **Dim_Date**, and **Dim_Geography** with the given structure.
2. Open **Microsoft Power BI Desktop**.
3. Click **Home → Get Data → Excel** and select the file.
4. Load all five tables into Power BI.
5. Go to **Model View** (Diagram icon on the left panel).
6. Identify the **Fact_Sales** table as the central fact table.
7. Drag **ProductKey** from Dim_Product to Fact_Sales to create a relationship.
8. Drag **CustomerKey** from Dim_Customer to Fact_Sales to create a relationship.
9. Drag **DateKey** from Dim_Date to Fact_Sales to create a relationship.
10. (Optional) Link **Dim_Geography** to Dim_Customer using GeoKey, if present.
11. Ensure all relationships are **One-to-Many** from Dimension to Fact table.
12. Verify the final model forms a **Star Schema** with Fact_Sales at the center and dimension tables around it.

OUTPUT:



7.

Aim:

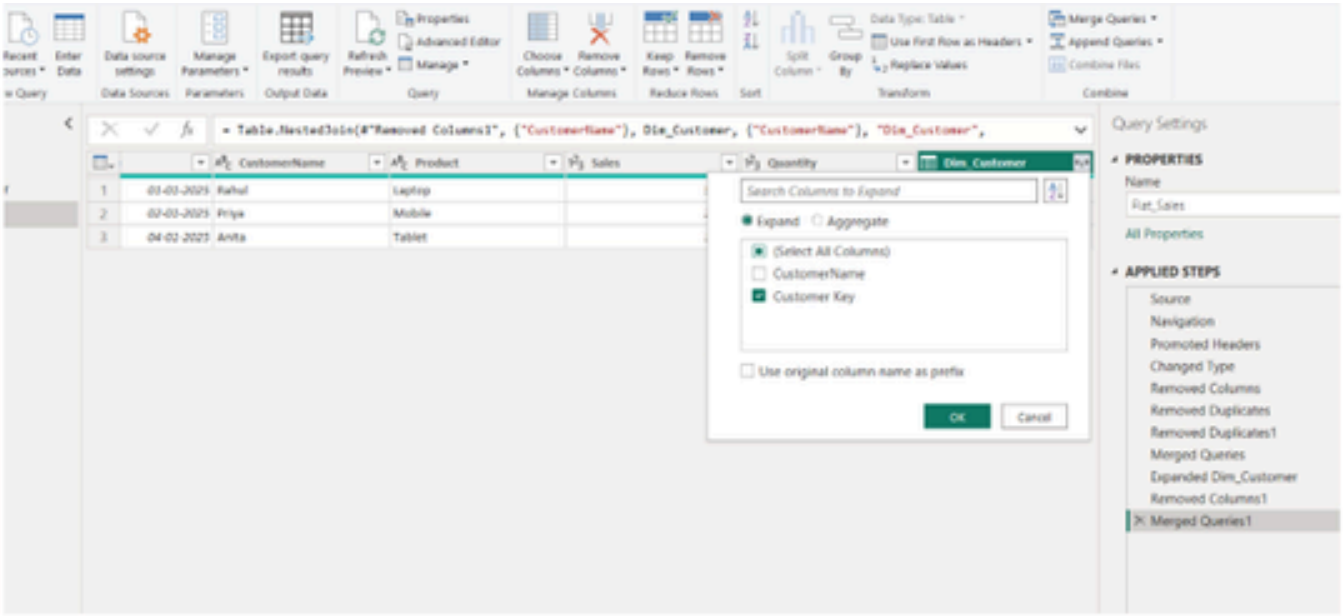
To normalize a flat CSV dataset in **Microsoft Power BI Desktop** by creating dimension and fact tables using Power Query, replacing text fields with keys, and exporting the normalized result.

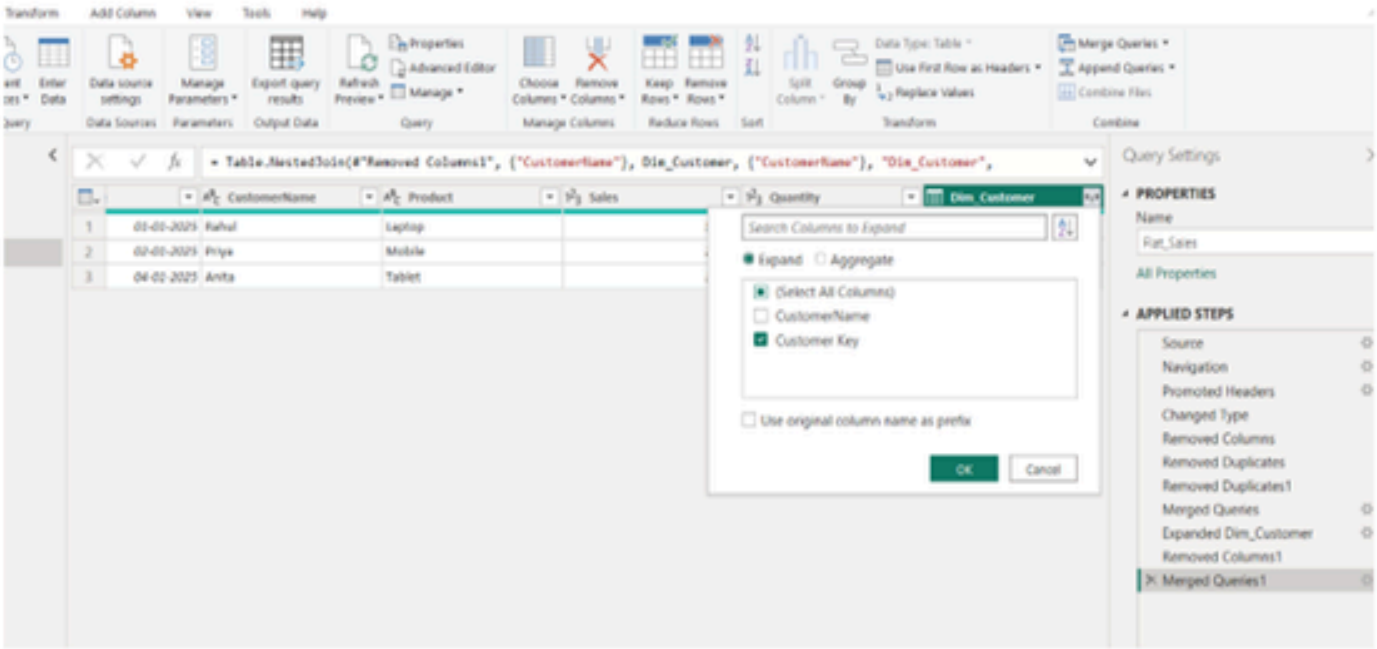
Algorithm (12 points):

1. Open **Microsoft Power BI Desktop**.
2. Click **Home → Get Data → Text/CSV** and select **Sales_Flat.csv**.
3. Click **Transform Data** to open **Power Query Editor**.
4. Right-click **Sales_Flat** query and choose **Duplicate** to create a new query.
5. Rename the duplicated query as **Dim_Customer**.
6. Keep only the **CustomerName** column and remove duplicates.
7. Add an **Index Column (From 1)** and rename it as **CustomerKey**.
8. Duplicate **Sales_Flat** again and rename it as **Fact_Sales**.

9. In **Fact_Sales**, keep required columns: OrderID, OrderDate, CustomerName, Product, Sales, Quantity.
10. Use **Merge Queries** to join **Fact_Sales** with **Dim_Customer** on CustomerName.
11. Expand and keep **CustomerKey**, then remove the CustomerName column to complete normalization.
12. Click **Close & Apply**, create a table visual, and **Export** the normalized data to CSV.

OUTPUT:





8.

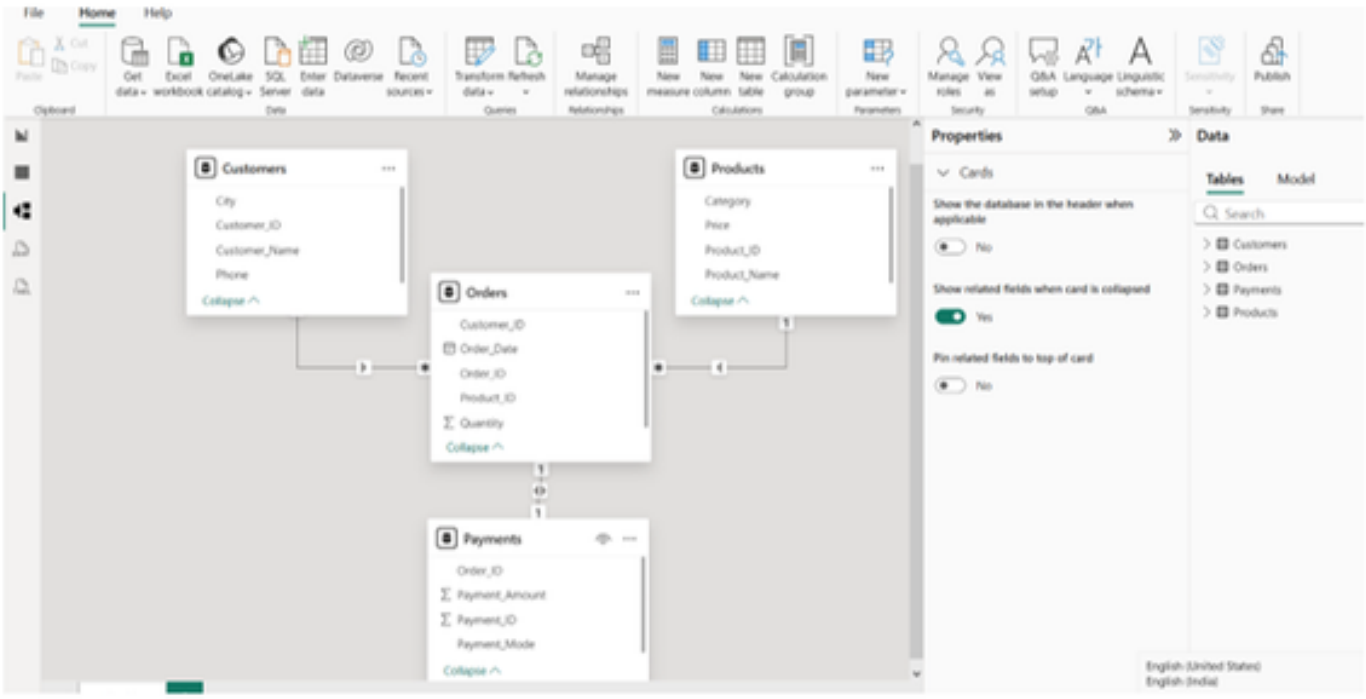
Aim:

To design and visualize an Entity-Relationship (ER) diagram and schema mockup by importing multiple related tables from Excel into Power BI and creating relationships using the Model View.

Algorithm (12 points):

1. Prepare an Excel file containing multiple related tables such as Customers, Products, Orders, and Payments with proper primary and foreign keys.
2. Open Power BI Desktop and select **Home → Get Data → Excel** to import the prepared Excel file.
3. Load all the sheets (tables) into Power BI Data Model.
4. After loading, navigate to the **Model View** to visualize the tables as entities.
5. Identify the **Primary Keys** in each table (Customer_ID, Product_ID, Order_ID, Payment_ID).
6. Identify the **Foreign Keys** used to connect the tables (Customer_ID in Orders, Product_ID in Orders, Order_ID in Payments).
7. Create relationships by dragging the foreign key column from one table to the corresponding primary key column in another table.
8. Set the **cardinality** of relationships appropriately (One-to-Many between Customers and Orders, Products and Orders, Orders and Payments).
9. Arrange the tables properly in the Model View to resemble a clear ER diagram structure.
10. Hide unnecessary columns and keep only important keys and attributes to make the schema clean and readable.
11. Rename tables and columns, if required, to follow proper database schema naming conventions.
12. Save the Power BI file and capture the Model View as the final ER diagram and schema mockup for documentation.

OUTPUT:



9.

Aim:

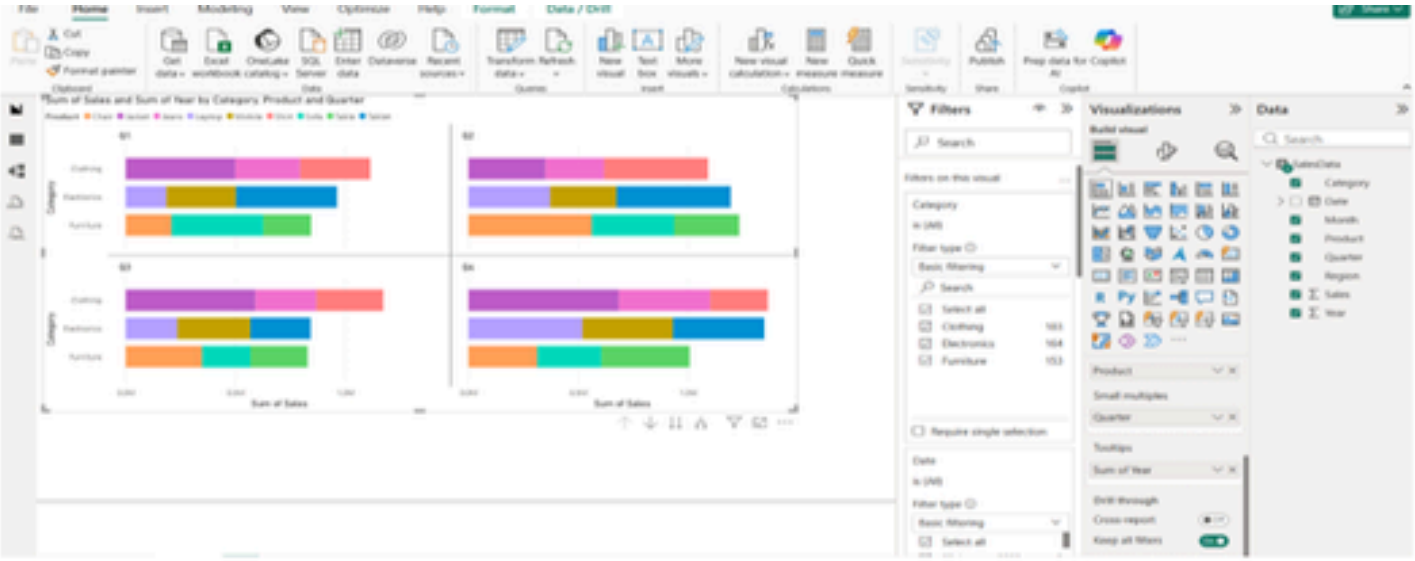
To apply and understand **Visual Level, Page Level, and Report Level Filters** in Power BI to control and analyze data effectively within a report.

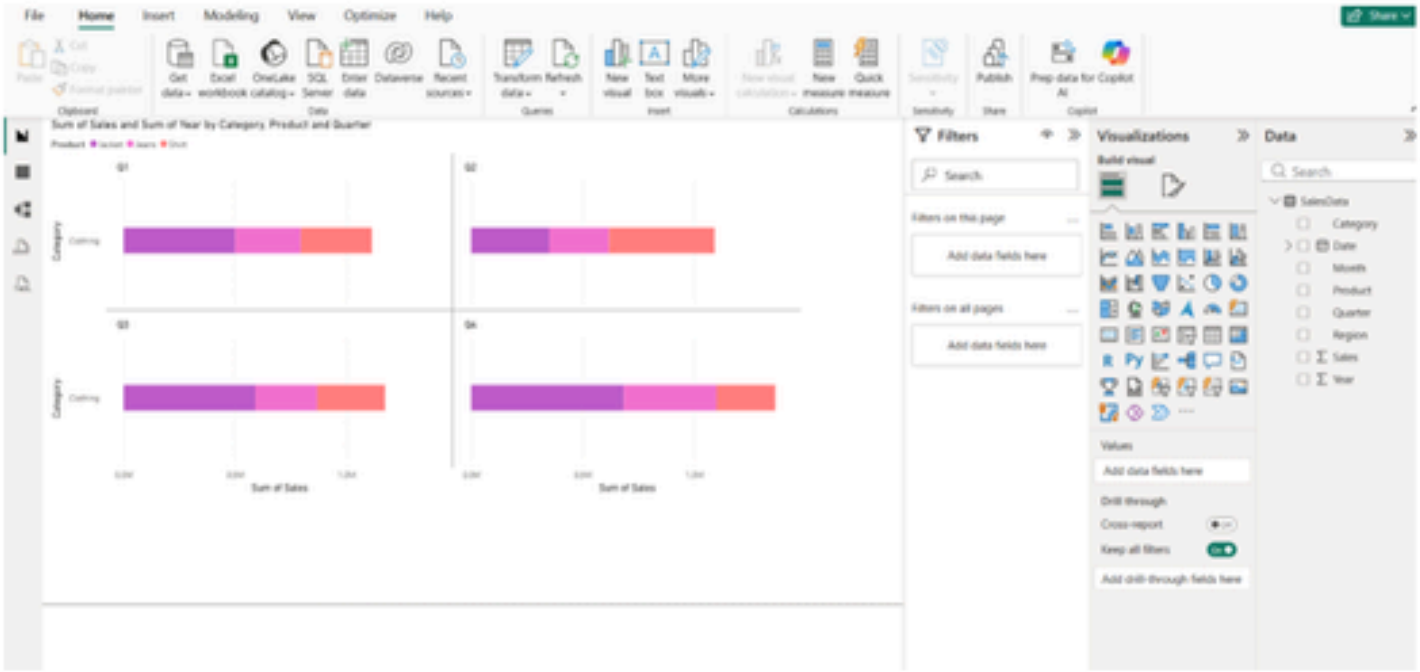
Algorithm (12 points):

1. Open **Power BI Desktop**.
2. Click on **Home → Get Data → Excel** and load the dataset *PowerBI_Filters_Slicers_Drill.xlsx*.
3. After loading, go to the **Report View**.
4. Create a **Bar Chart** visual to display *Sales by Product*.
5. Select the Bar Chart and open the **Filters pane**.
6. Drag the **Category** field into the **Visual Level Filters** section.
7. Select only **Electronics** to filter the bar chart.
8. In the Filters pane, drag the **Region** field into the **Page Level Filters** section.
9. Select only **North** and **East** regions to filter the entire page.
10. Drag the **Year** field into the **Report Level Filters** section.
11. Select the year **2024** to filter the whole report.

12. Observe how each filter affects the visual, page, and report differently and verify the filtered results using a Table visual.

OUTPUT:





10.

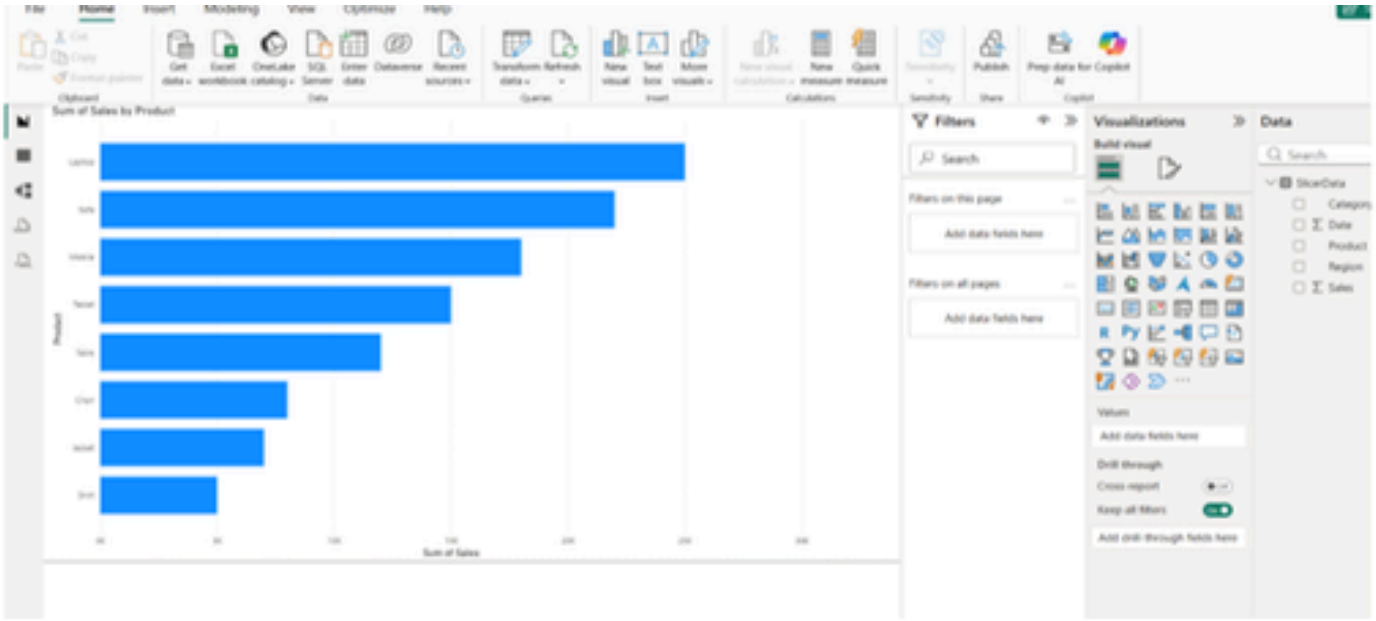
Aim:

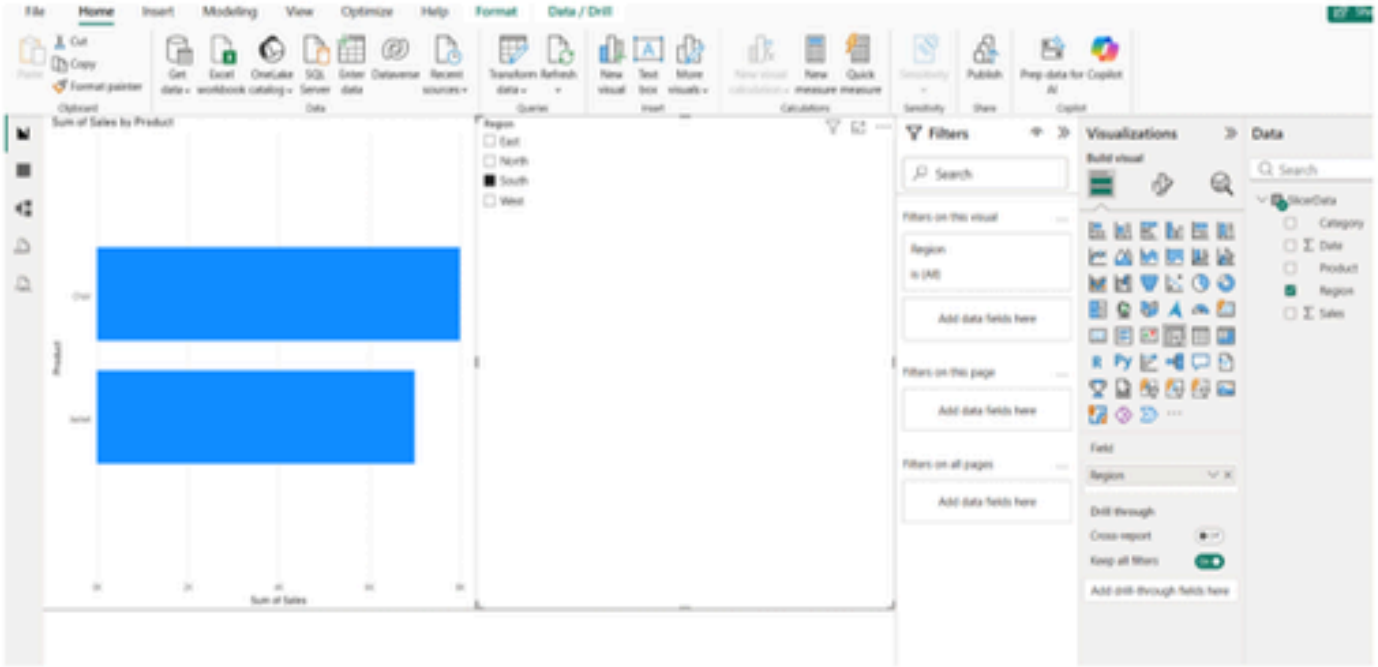
To create and use a **Slicer** in Power BI to interactively filter and analyze sales data based on Region.

Algorithm (12 points):

1. Open **Power BI Desktop**.
2. Click **Home → Get Data → Excel**.
3. Select the file **PowerBI_Slicer_Practice.xlsx** and click **Load**.
4. After loading, go to the **Report View**.
5. Insert a **Bar Chart** from the Visualizations pane.
6. Drag the **Product** field to the **Axis** area.
7. Drag the **Sales** field to the **Values** area.
8. Click on a blank space in the report canvas.
9. Select the **Slicer** visual from the Visualizations pane.
10. Drag the **Region** field into the Slicer.
11. Click different region values (North, South, East, West) and observe the changes in the bar chart.

OUTPUT:





11.

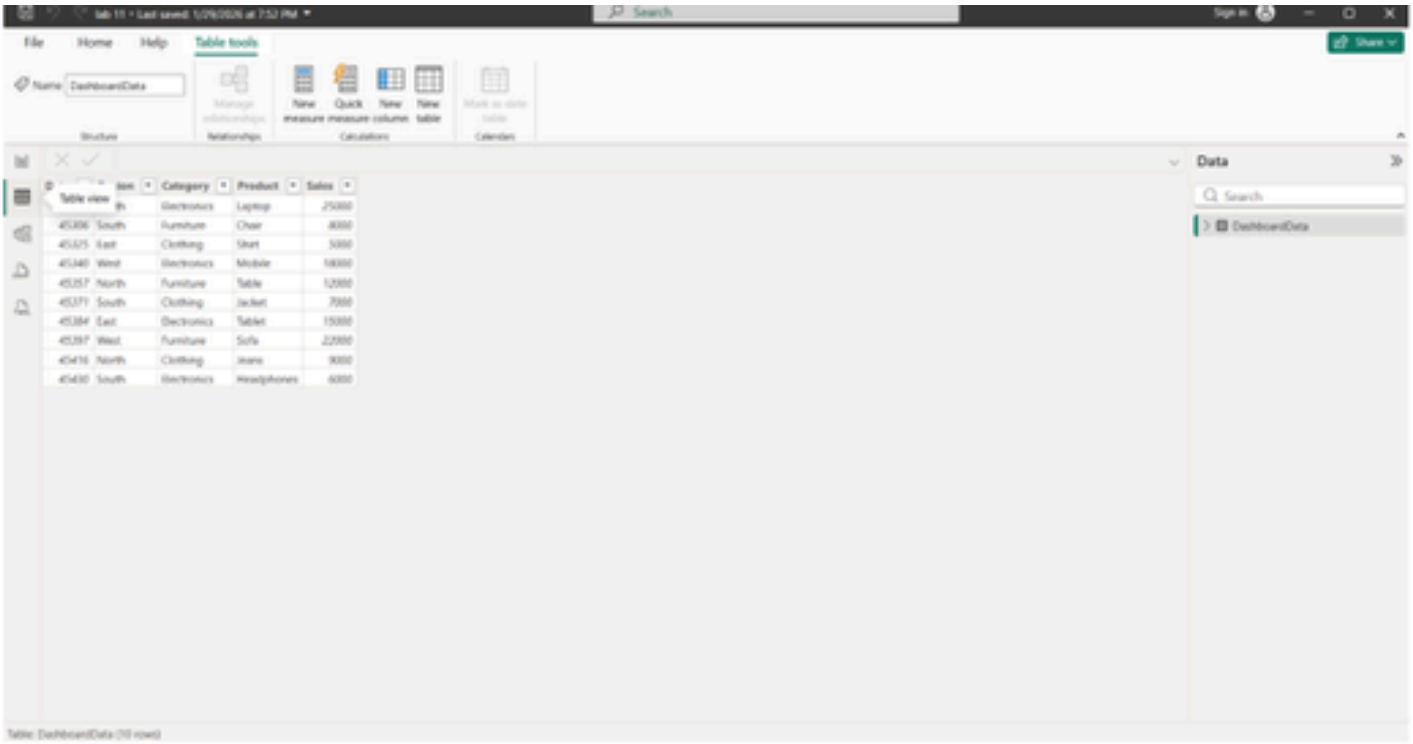
Aim:

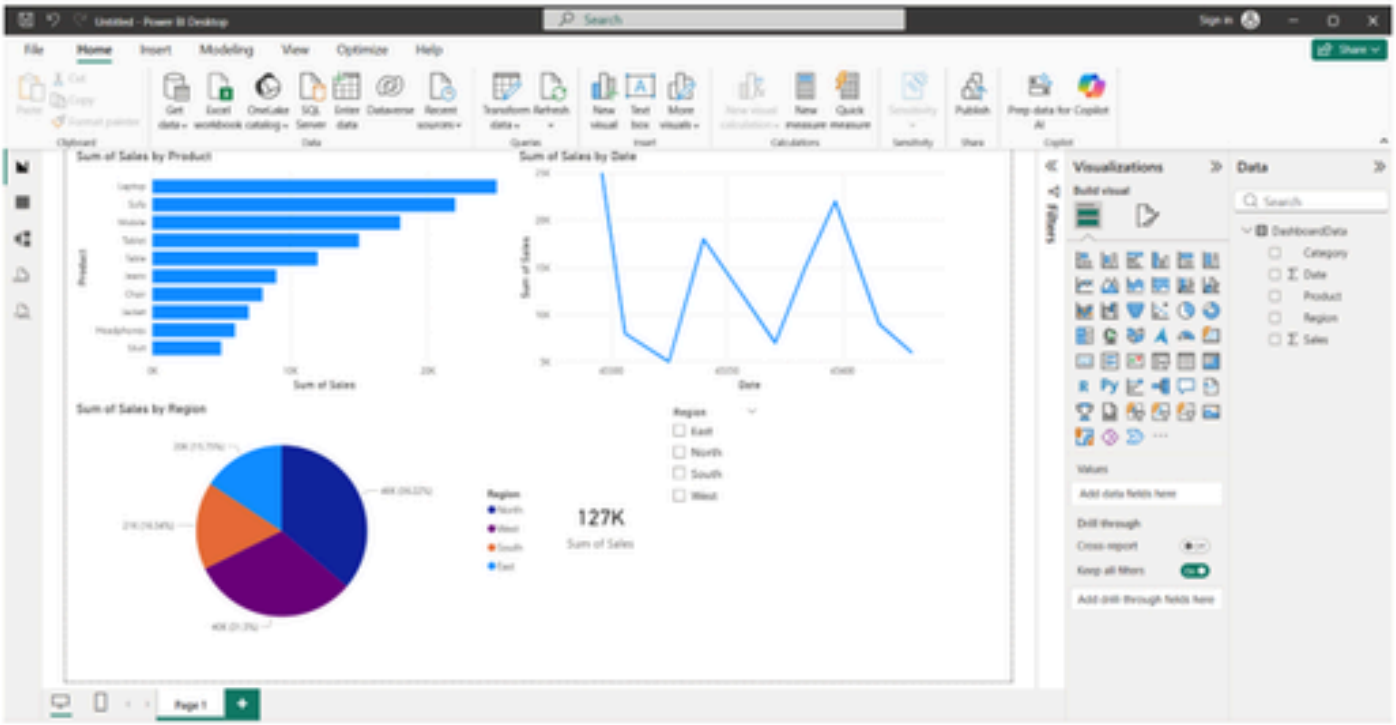
To import prepared Excel data into Power BI and design an interactive dashboard using different visuals and a slicer to analyze sales performance.

Algorithm (12 points):

1. Open **Power BI Desktop**.
2. Click **Home → Get Data → Excel**.
3. Select the file **PowerBI_Dashboard_Practice.xlsx** and click **Load**.
4. Go to the **Report View** after the data is loaded.
5. Insert a **Bar Chart** and drag **Product** to Axis and **Sales** to Values.
6. Insert a **Pie Chart** and drag **Region** to Legend and **Sales** to Values.
7. Insert a **Line Chart** and drag **Date/Month** to Axis and **Sales** to Values.
8. Insert a **Card** visual and drag **Sales** to display Total Sales.
9. Click on a blank area and insert a **Slicer** visual.
10. Drag the **Region** field into the Slicer to enable filtering.
11. Arrange all visuals neatly and provide suitable titles for each chart.
12. Test the dashboard by selecting different regions in the slicer and observe the dynamic changes in all visuals.

OUTPUT:





12.

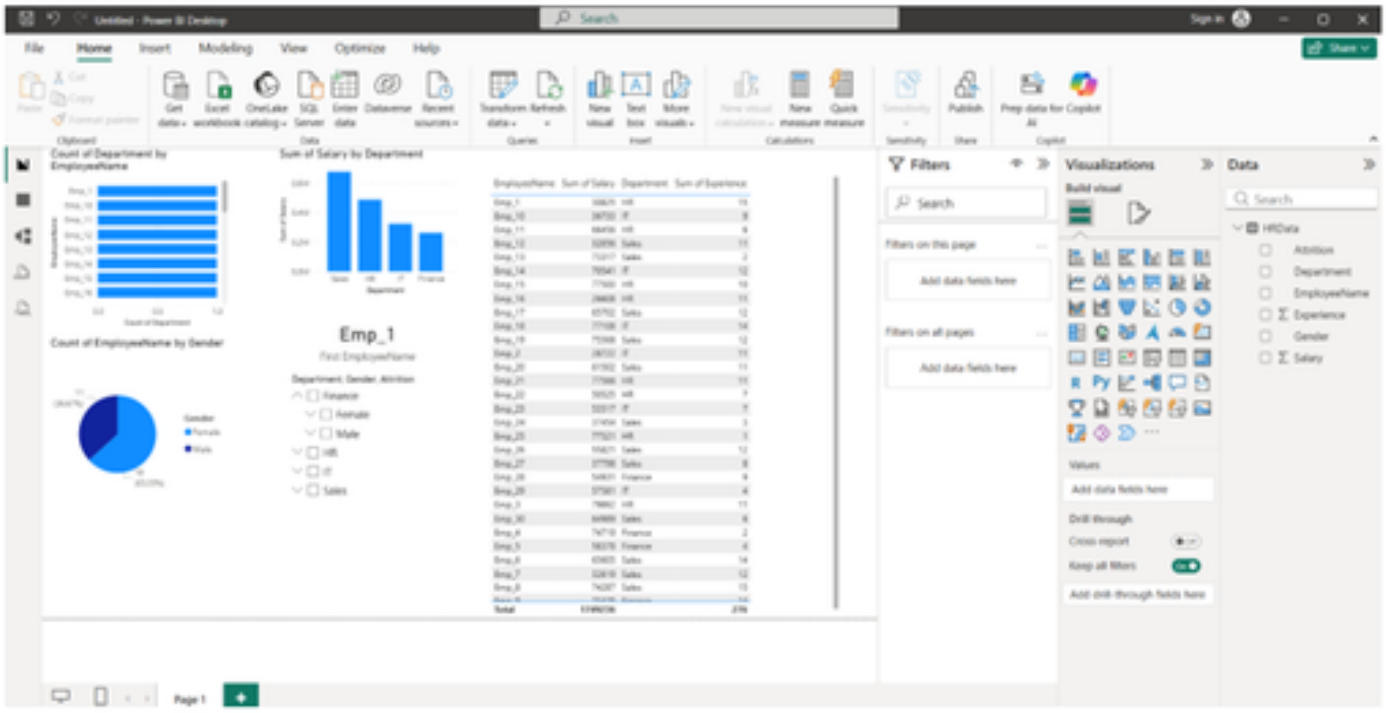
Aim:

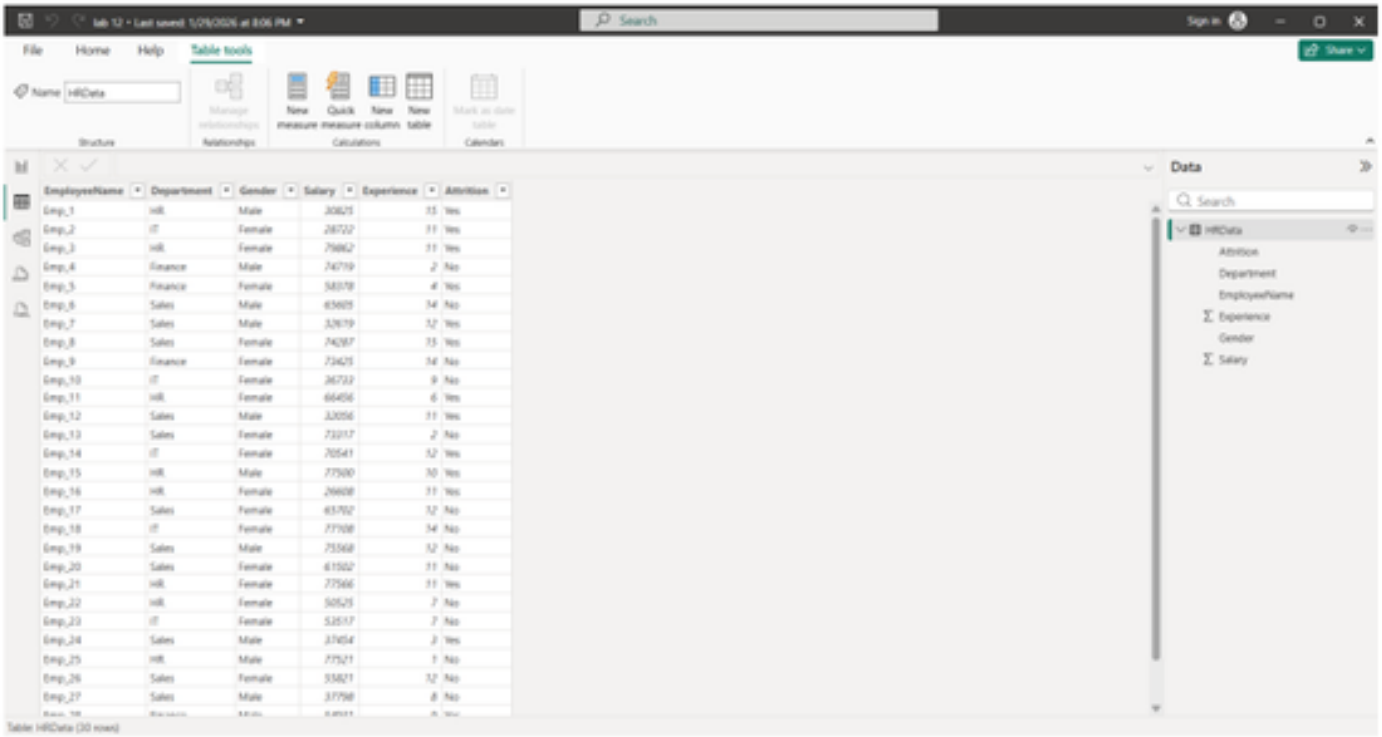
To create an interactive HR dashboard in Power BI to analyze employee distribution, salary, gender ratio, and attrition using visuals and slicers.

Algorithm (12 points):

1. Open **Power BI Desktop**.
2. Click **Home → Get Data → Excel**.
3. Select **PowerBI_HR_Dashboard.xlsx** and click **Load**.
4. Go to **Report View**.
5. Insert a **Bar Chart** and place **Department → Axis**, **EmployeeName → Count**.
6. Insert a **Pie Chart** and place **Gender → Legend**, **EmployeeName → Count**.
7. Insert a **Column Chart** and place **Department → Axis**, **Salary → Values**.
8. Insert a **Card** visual and place **EmployeeName → Count**.
9. Insert a **Table** visual with EmployeeName, Department, Salary, and Experience.
10. Add a **Slicer** and drag **Department** into it.
11. Add a **Slicer** and drag **Gender** into it.
12. Add a **Slicer** for **Attrition** and test the dashboard interactivity.

OUTPUT:





13.

Aim:

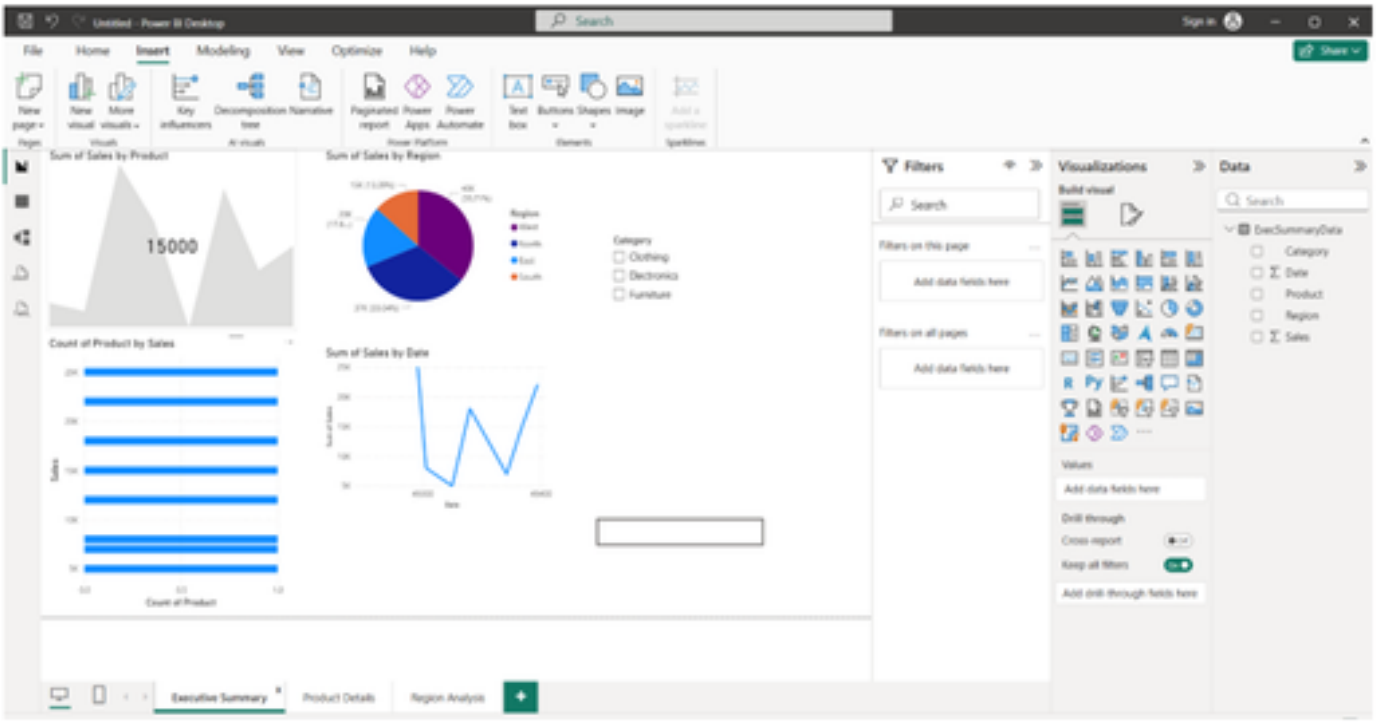
To create an Executive Summary page and implement a Visual Walkthrough in Power BI using multiple report pages, visuals, slicers, and navigation buttons for effective data presentation

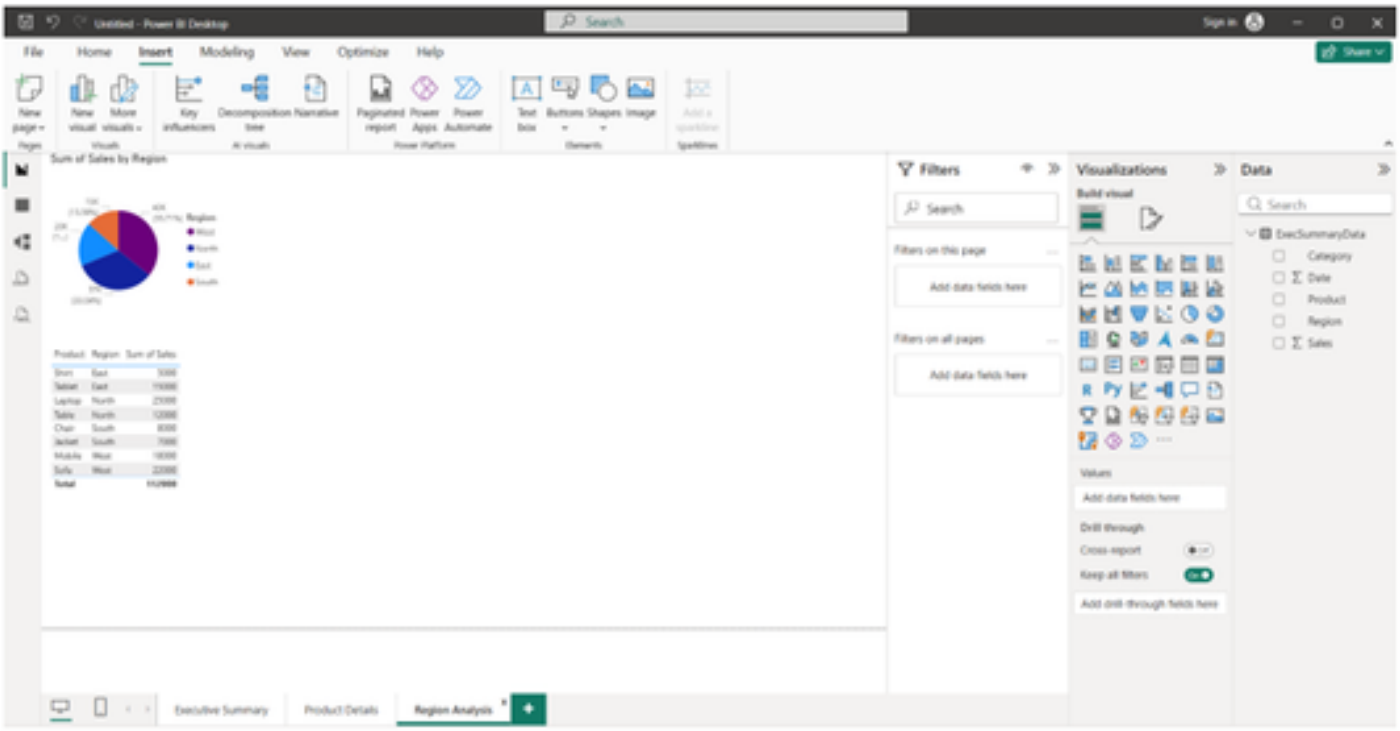
Algorithm (12 points):

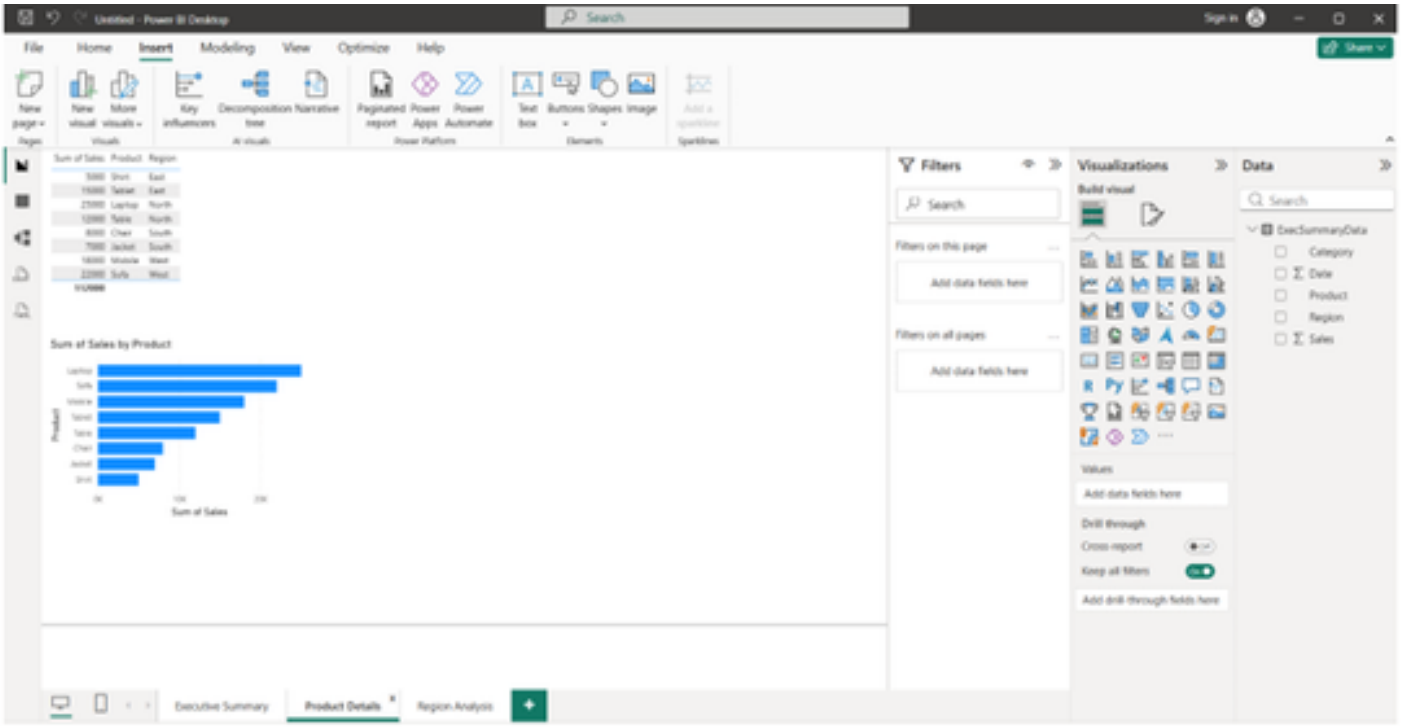
1. Open **Power BI Desktop**.
2. Click **Home → Get Data → Excel** and load *PowerBI_Executive_Summary_Practice.xlsx*.
3. Rename the first report page as **Executive Summary**.
4. Insert **Card visuals** to display Total Sales and Count of Products.
5. Insert a **Bar Chart** with Product on Axis and Sales on Values.
6. Insert a **Pie Chart** with Region on Legend and Sales on Values.
7. Insert a **Line Chart** with Date on Axis and Sales on Values.
8. Add a **Slicer** and drag Category into it for interactivity.
9. Add a new page and rename it **Product Details**.
10. Insert a **Table** and Bar Chart to show detailed Product and Sales information.

11. Add another page named **Region Analysis** and create visuals for Region-wise sales.
12. Insert **Buttons** with Page Navigation to move between pages, creating a Visual Walkthrough of the report.

OUTPUT:







14.

Aim:

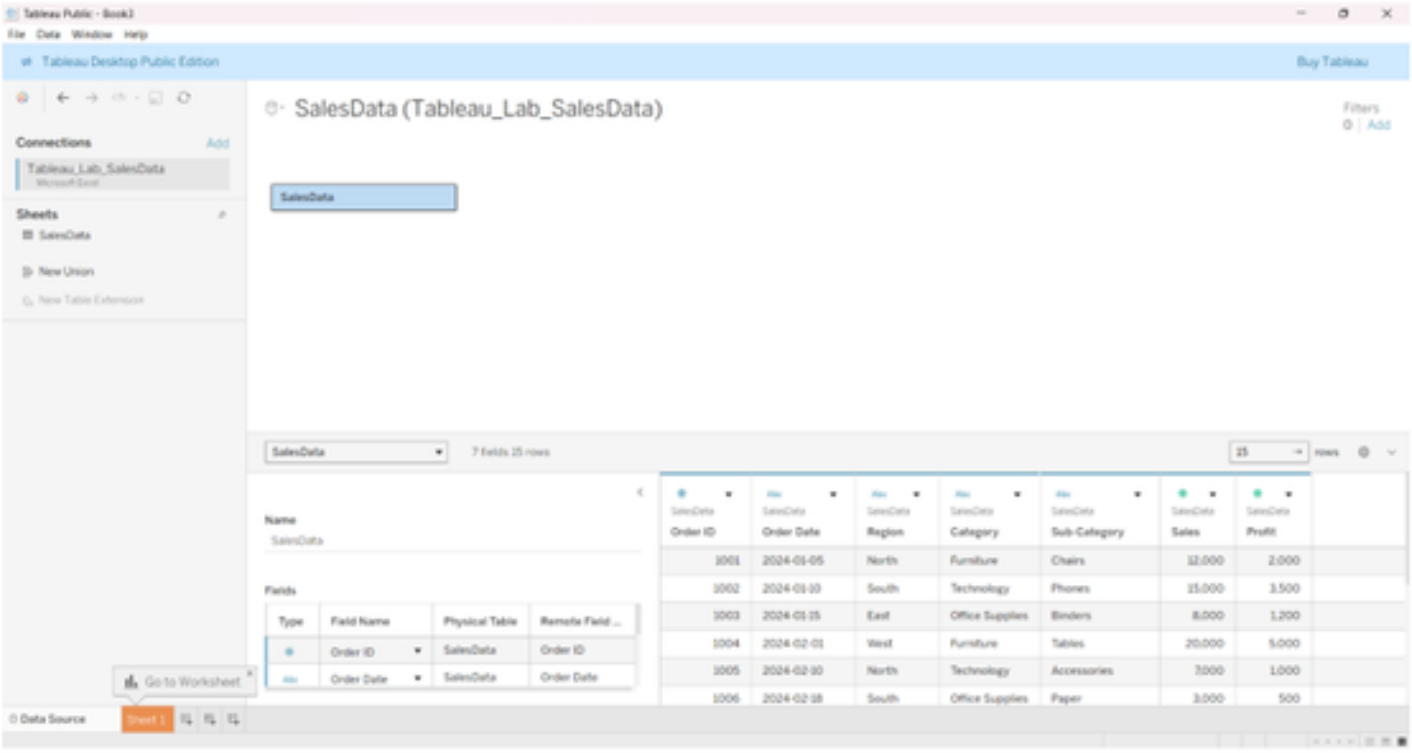
To connect an Excel dataset to Tableau and create different visualizations (Bar chart, Line chart, Pie chart, Map) and combine them into an interactive dashboard using filters.

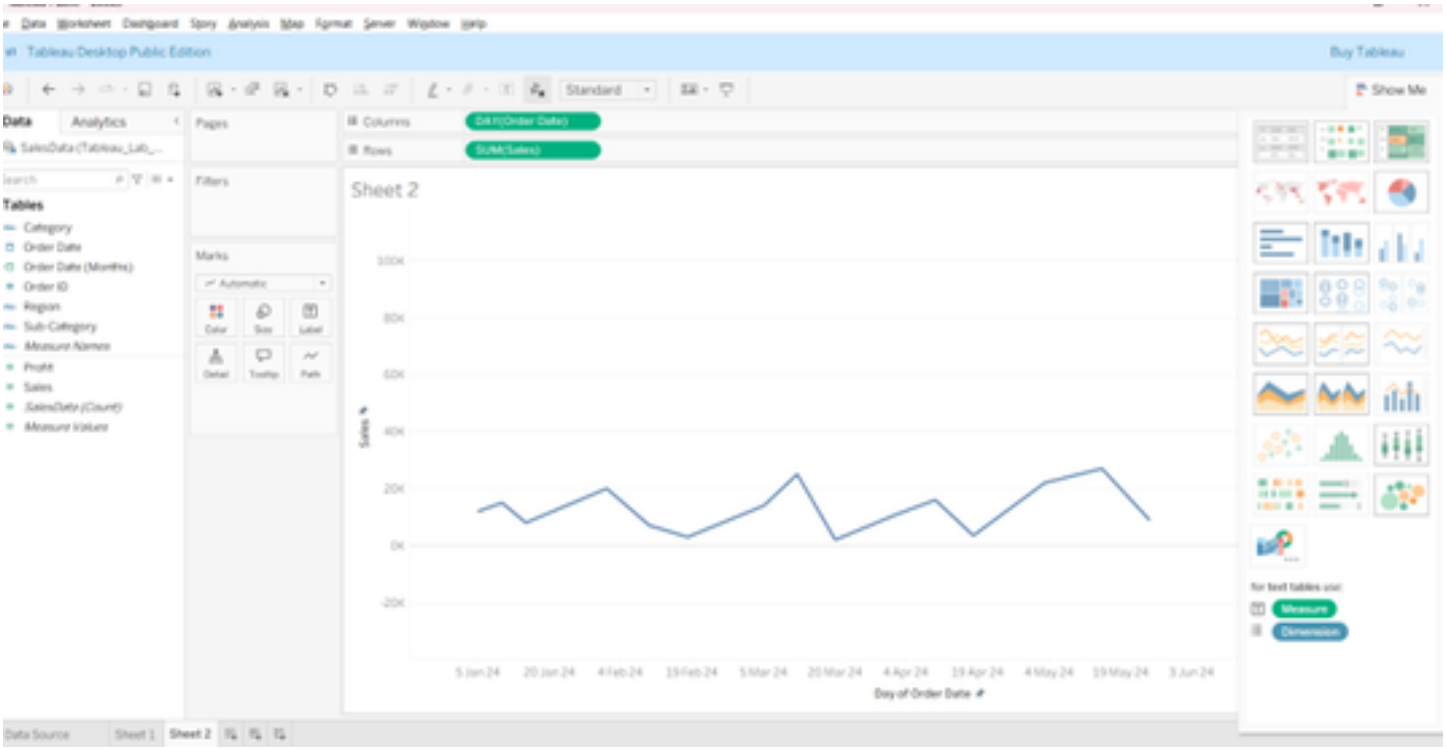
Algorithm (12 points):

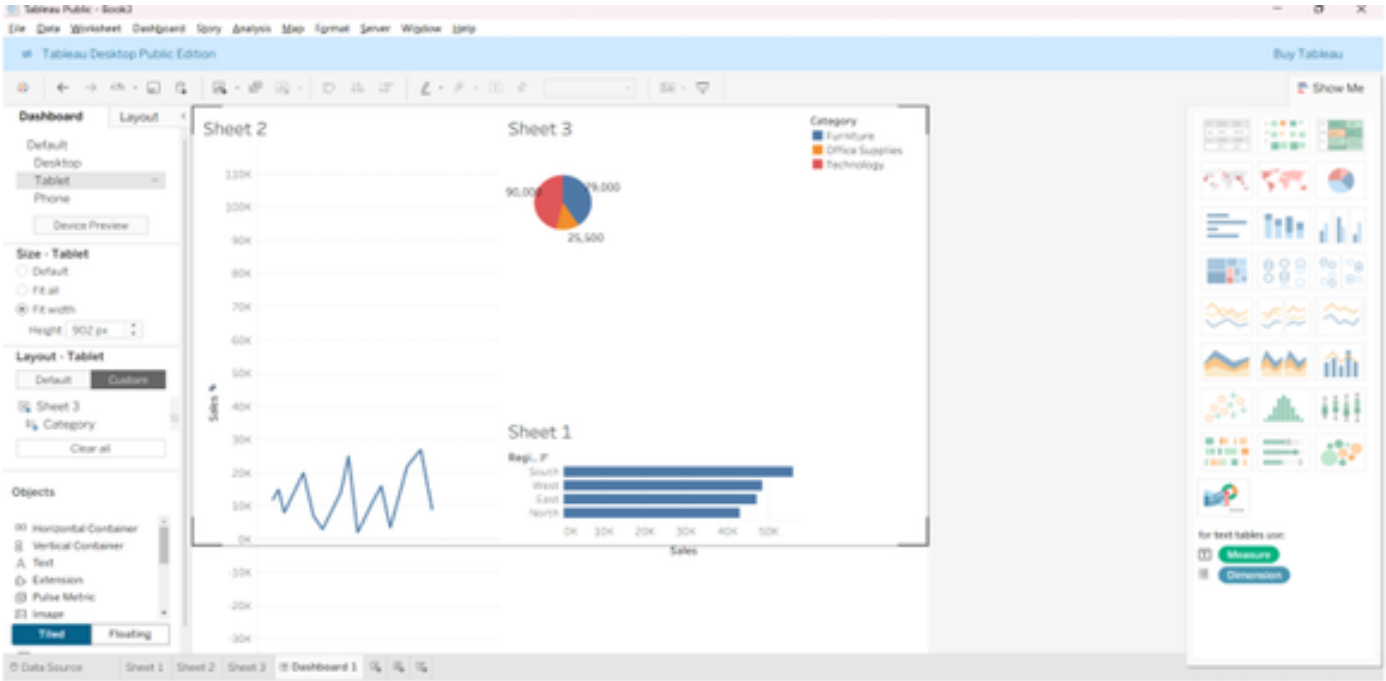
1. Start the Tableau Desktop application.
2. Select **Microsoft Excel** from the Connect panel.
3. Browse and choose the file **Tableau_Lab_SalesData.xlsx**.
4. Load the Excel file into Tableau by opening the data source page.
5. Drag the **SalesData** worksheet to the data canvas.
6. Verify that all fields are loaded correctly in the Data pane.
7. Click on **Sheet 1** to begin visualization.
8. Drag the **Region** dimension to the **Rows** shelf.
9. Drag the **Sales** measure to the **Columns** shelf.
10. Observe the automatically generated bar chart.
11. Sort the bar chart in descending order based on sales values.

12. Rename the worksheet as **Sales by Region**.

OUTPUT:







15.

Aim:

To create new calculated fields in Tableau and use them in visualizations and dashboards.

Algorithm (12 points):

1. Open Tableau Desktop.
2. Connect the Excel file to Tableau.
3. Load the dataset into the worksheet.
4. Create a calculated field **Total Amount**.
5. Apply formula Quantity × Unit Price.
6. Create calculated field **Discount** using IF condition.
7. Create calculated field **Final Amount**.
8. Drag Region and Final Amount to create bar chart.
9. Create line chart using Order Date and Final Amount.
10. Apply Region as filter.
11. Show filter for interaction.
12. Create dashboard combining charts.

OUTPUT:

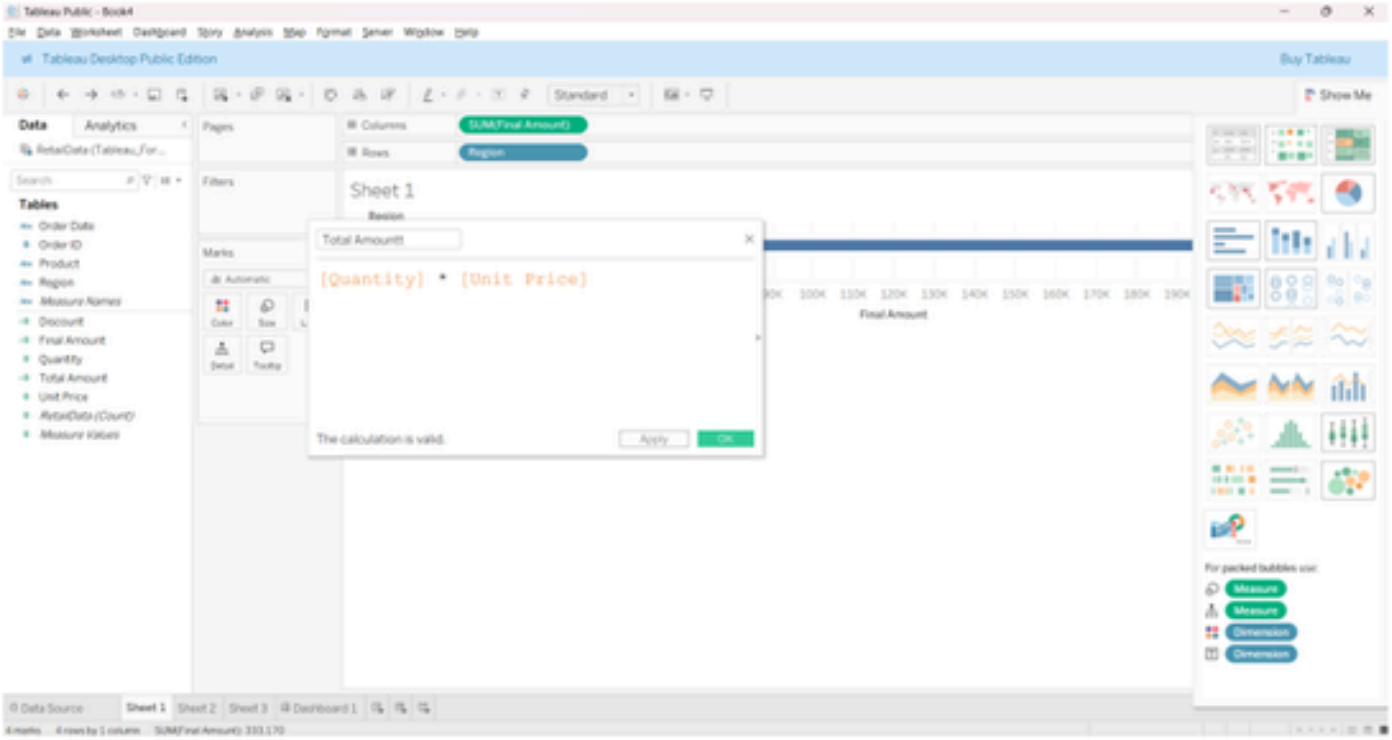


Tableau Desktop Public Edition
Buy Tableau
Show Me
Data
Analytics
Tables
Order Date
Order ID
Product
Region
Measure Names
Discount
Final Amount
Quantity
Total Amount
Unit Price
Pages
Filters
Marks
Columns
SUM(Final Amount)
Rows
Region
Sheet 1
Total Amount
[Quantity] * [Unit Price]
The calculation is valid.
Apply
OK
Final Amount
Data Source
Sheet 1
Sheet 2
Sheet 3
Dashboard 1

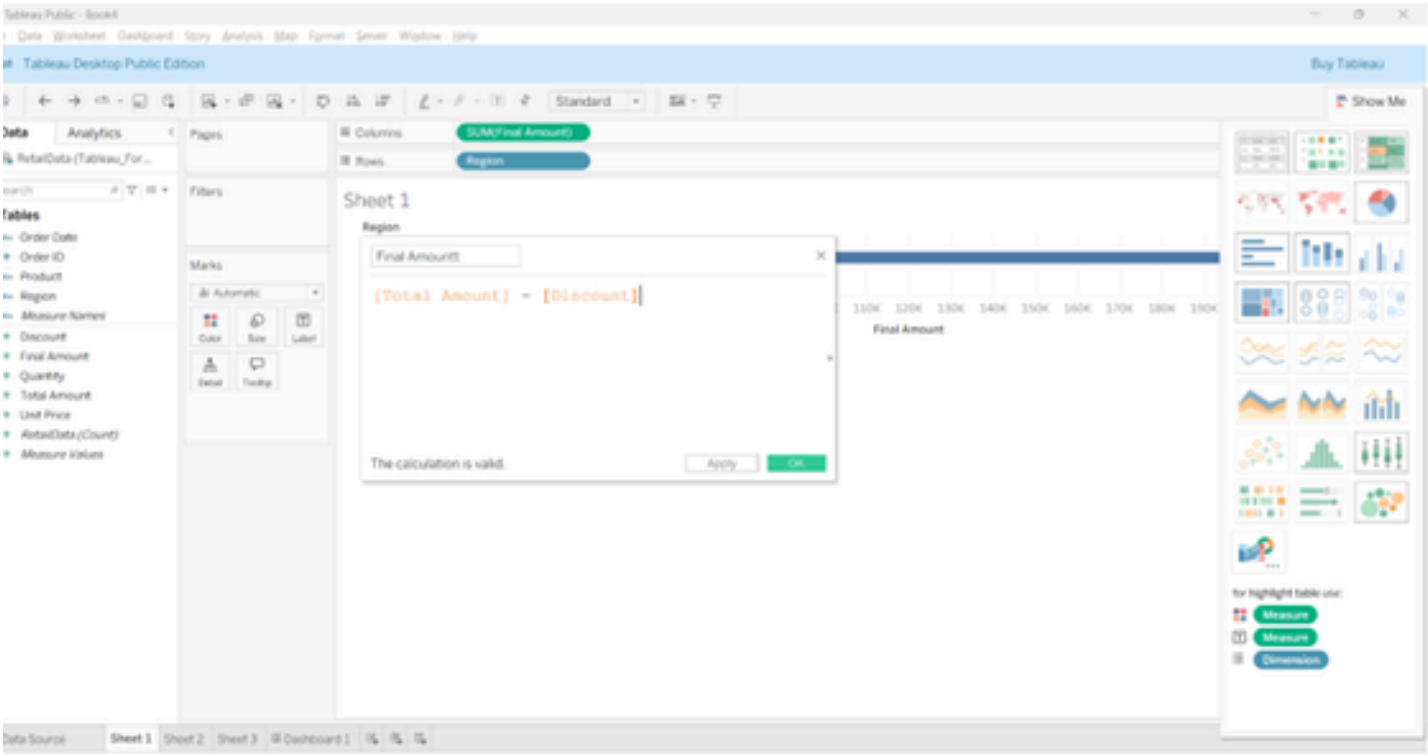


Tableau Desktop Public Edition
Buy Tableau
Show Me
Data
Analytics
Tables
Order Date
Order ID
Product
Region
Measure Names
Discount
Final Amount
Quantity
Total Amount
Unit Price
Pages
Filters
Marks
Columns
SUM(Final Amount)
Rows
Region
Sheet 1
Final Amount
[Total Amount] - [Discount]
The calculation is valid.
Apply
OK
Final Amount
Data Source
Sheet 1
Sheet 2
Sheet 3
Dashboard 1